

Appendices:

Appendix A: Personas

Personas are defined as fictional representations of users aimed to represent the diverse range of individuals interacting with a specific application. They are used in order to observe and simulate a user’s interaction with the platform once completed. Ultimately, constructing different personas can help identify weak points and edge cases within the platform. In our case, all personas are assumed wto be unfamiliar with any programming knowledge.

A.1 Tourist/Traveller

This persona represents a typical travel enthusiast, visiting different countries and attractions. Each tourist as a bucket-list with attractions they decide to visit for that country. Tourists interact with each other, with guides, obtain discounts and discover new attractions.

A.2 Local Business Owner (Sponsor/Business Partner)

Since the app features discounts for various restaurants or souvenir shops, this persona represents local shop-owners that wish to improve their business. They process the monetary fee through the platform and in return offer the tourists a discount, voucher or even a free souvenir. Furthermore, they are also considered local sponsors since they ultimately wish for publicity for their business.

A.3 Travel Agent

This persona represents a travelling expert with extended knowledge about various attractions. They help clients plan, book, and customize trips by offering professional advice on destinations, accommodations, and experiences.

A.4 (Potential) External Sponsor

A sponsor is typically a brand, or local business which seeks visibility for their product among any individuals. They want to attract and engage individuals through gamified experiences and offered promotions in exchange for brand exposure.

Appendix B: User Stories

A user story is a brief description of a feature that a persona requires in the system. Furthermore, the overall benefit of this feature for the system’s development is also presented.

User Story ID	User Story
US1	As a Tourist, I wish to be able to see information for each attraction.
US2	As a Tourist, I wish to be able to answer cultural questions about an attraction to improve my knowledge and receive points for completing a quiz per attraction.

User Story ID	User Story
US3	As a Tourist, I wish to be able to participate in a country's league for the ultimate travelling champion.
US4	As a Tourist, I want to engage with other travellers through a community platform.
US5	As a Tourist, I want to be able to suggest new attractions in a country and earn points if my suggestion is validated.
US6	As a Tourist, I wish to be able mark attractions as visited and cross them from my bucket-list.
US7	As a Tourist, I want the platform to create a travelling plan for me, which I then will be able to export the external map applications.
US8	As a Tourist, I want the platform to suggest nice local and traditional restaurants and open-area places when I am nearby and attraction.
US9	As a Tourist, I want to pay extra for this platform to suggest me available accommodations near my current attraction that are also not crowded.
US10	As a Tourist, I want to pay extra to remove pop-up ads.
US11	As a Local Business Owner, I want the platform to recommend my business through pop-up ads.
US12	As a Local Business Owner and Sponsor, I want the platform to offer 5% discounts for all platform users to increase publicity.
US13	As a Local Business Owner, I want the free earned souvenir to have a representative picture on the platform.
US14	As a Local French Restaurant Owner, I want the platform to recommend my business at the end of the quizzes for traditional places.
US15	As a Travel Agent, I want offer city tours through TravelGo, for my clients to play along and for me to stand out from standard booking platforms.
US16	As a Travel Agent, I want the platform to highlight premium guided tours or special events in collaboration with my agency in order to promote my business.
US17	As an External Sponsor, I wish to financially support the platform's development in exchange for exposure.

Appendix C: Recommended Architectural Patterns

C.1 CQRS (Command Query Responsibility Segregation)

Implementing CQRS would be beneficial for TravelGo, since it improves scalability, modularity and performance. Because the write side of the system would be separated from the read side, we can use a different model for the reads, which makes the read side a lot faster. This is advantageous since our system will have far more reads than writes, since our users will mainly use the map feature with all of the tourist attractions. It is also a lot easier to scale the read and write sides when they are independent from each other.

Another benefit is that the data is easier to update, which improves the modularity of the system. This is especially important since we often have to add or change the data of the tourist attractions. Lastly, CQRS fits really well with event driven communication, especially event sourcing, which we plan to use.

C.2 Circuit Breaker

In a distributed microservices system, the Circuit Breaker pattern acts like a protective mechanism ensuring that when a service begins failing or unresponsive, the circuit breaker disconnects it from the rest of the infrastructure and stops forwarding further calls. This approach is known as fast, but gracefully failing and it is preferred over waiting or retrying endlessly. While the circuit is open, requests immediately return an error or fallback without following the entire failing service. The pattern thus protects against cascading failures and keeps the resources from being exhausted early.

This pattern is a great fit since TravelGo depends on multiple remote services (maps, payments, rewards, tourism/attractions data, leaderboards) and it would mainly be used on the server side between services and third-party APIs. In the case of the rewards or payment systems, for example, if the services are down, the platform detects repeated failures and stops sending more requests to that service.

Below is a table showcasing appropriate architectural patterns for the implementation of the TravelGo system. The research for suitable microservice patterns was heavily relying on Ahmad's article [\[10\]](#).

Pattern Name	Pattern Description
API Gateway	Serves as a single entry point for client requests, routing them to appropriate microservices and safely manages concerns such as authentication.
Service Discovery (Service Registry)	Allows microservices to dynamically find and communicate with each other by registering themselves and maintaining a central registry.
Circuit Breaker	Prevents cascading failures such that each service is independently connected and in case of failure the entire system remains running with only the failing one being shut down.
Retry Pattern	Automatically retries failed operations in an attempt to fix errors.
Event Sourcing Pattern	Stores state changes as a sequence of events which can be replayed to reconstruct the system's state at any point in time.
Event driven	Uses events as the primary means of communication between services, allowing for asynchronous actions.
CQRS	Separates read and write operations into independent models to optimize performance, scalability, and complexity.
Sidecar Pattern	The system deploys auxiliary components alongside their primary service to account for logging or configuration concerns.
"Smart Endpoints, Dumb Pipes"	Ensures that the logic is maintained in the services only and their connections (pipes) remain with little to no complexity implemented.

Pattern Name	Pattern Description
Shadow Deployment	Runs a new version of a service in parallel (a "shadow") without affecting user and or the network's traffic, allowing for easier testing.
Stateless Services	Ensures that services do not store the state of client sessions locally, thus enabling them to be scaled and replaced independently.

Appendix D: Component View

D.1 Frontend Service

Responsible for rendering the user interface and providing interaction between the user and backend services.

Main Components:

- UI Layer (HTML/CSS/JS, Leaflet.js): Displays maps, attractions, posts, leaderboards, and chats interactively.
- View Controller: Routes page navigation between Home, Posts, Leaderboard, and Chat.
- API Client: Handles all REST API requests to the API Gateway.
- Session Manager: Manages user sessions and local caching for efficiency.

Responsibilities:

- Render the interactive map on the home page.
- Display posts, and leaderboard fetched from backend services.
- Enable communication via chat and user-generated content creation.

D.2 API Gateway

Serves as the single point of entry for all client requests. It provides request routing, authentication, and load balancing, ensuring a unified interface between the frontend and backend microservices.

Main Components:

- Request Router: Forwards incoming API calls to appropriate microservices.
- Authentication Handler: Verifies tokens and manages session integrity.
- Load Balancer: Distributes requests across instances of scalable services.
- Response Aggregator: Merges responses from multiple microservices into unified responses for the client.

Responsibilities:

- Ensure secure and efficient request dispatching.
- Hide service complexity from the client.
- Facilitate scalability by abstracting backend endpoints.

D.3 User Service

Manages all user related operations including authentication, profile management, preferences, and subscription handling.

Main Components:

- UserController: Exposes APIs for login, registration, and profile updates.
- UserManager: Implements business logic for user authentication, account status, and preferences.
- SubscriptionHandler: Manages premium user subscriptions and payment verification.
- UserRepository: Handles persistent data storage in the database.

Responsibilities:

- Authenticate users and maintain secure sessions.
- Manage personal information, interests, and privacy preferences.
- Interface with payment services for premium access.

D.4 Map Service

Integrates external map APIs and manages attraction visualisation.

Main Components:

- MapController: Provides APIs for fetching map tiles and location data.
- AttractionLocator: Retrieves nearby attractions using geospatial queries.
- MapIntegrator: Connects with third-party providers (Leaflet, OpenStreetMap).

Responsibilities:

- Provide up-to-date map data for visualisation.
- Support attraction overlays and filtering by category.
- Handle external API calls efficiently to reduce latency.

D.5 Posts Service

Allows users to share travel experiences, images, and recommendations about attractions. It is event-driven and interacts with the Leaderboard Service through Kafka.

Main Components:

- PostController: Handles CRUD operations for posts and attractions.
- PostManager: Contains business rules for validating and storing content.
- AttractionRepository: Manages data persistence for attractions linked to posts.
- EventPublisher: Publishes "PostCreated" events to Kafka for processing.

Responsibilities:

- Facilitate creation and retrieval of travel-related posts.
- Maintain data integrity between posts and attractions.
- Trigger leaderboard updates via Kafka when users posts.

D.6 Leaderboard Service

Calculates user rankings and scores based on engagement activities.

Main Components:

- **LeaderboardController:** Exposes APIs for fetching rankings.
- **ScoreCalculator:** Computes scores and rankings dynamically based on user activity.
- **EventHandler:** Subscribes to Kafka events like "PostCreated" to update scores.
- **LeaderboardRepository:** Stores user score histories and current rankings.

Responsibilities:

- Aggregate activity data into user scores.
- Expose leaderboards for global and local rankings.

D.7 Chat Service

Supports real-time text communication among users for travel discussions, experience sharing, and community engagement.

Main Components:

- **ChatController:** Provides WebSocket endpoints for real-time messaging.
- **MessageBroker:** Handles routing and delivery of messages between users.
- **ChatRepository:** Persists chat history and user threads.
- **NotificationManager:** Pushes chat notifications to users via event triggers.

Responsibilities:

- Enable scalable, low-latency real-time communication.
- Persist messages and ensure reliable delivery.
- Integrate with user profiles and community features.

D.8 Kafka

Serves as the central asynchronous event broker, decoupling microservices and enabling real-time communication.

Main Components:

- **Producer:** Sends messages (events) from publishing services (e.g., Post Service).
- **Consumer:** Listens to and processes events in subscribed services (e.g., Leaderboard Service).
- **Topic Manager:** Organises event topics and partitions for scalability.
- **Offset Manager:** Tracks message consumption state to ensure reliability.

Responsibilities:

- Enable real-time event propagation.
- Ensure high throughput and scalability in communication.
- Decouple service dependencies to reduce coupling and improve modularity.

D.9 Database Layer

Consists of multiple logical databases, each associated with a microservice to ensure autonomy and data encapsulation.

Main Components:

- UserDB: Stores user credentials and profiles.
- MapDB: Stores attraction data.
- PostDB: Stores posts.
- LeaderboardDB: Stores user scores and rankings.
- ChatDB: Persists messages and user threads.

Each database can scale horizontally and uses backups for ensuring data integrity and fault tolerance.

Appendix E: Acknowledgement of AI

The architects created all of the information in this report. However, sections were rewritten for clarity and conciseness with the help of generative AI. "Rewrite this paragraph/sentence such that it is easier to understand" was the standard query that was employed. After that, the results were examined and used as a guide to revise certain sections of the report. The architect personally rewrote the content in their own terms rather than merely copying it from the AI model.